

An implementation of Open-RMF in an industrial environment

Michele Belletti

2025
December

Contents

Part I

Introduction

This work is a study on a new system OPEN-RMF that is still in active development but has reached a good state to apply for the robots in the facility Elettra Sincrotrone Trieste

Chapter 1

Background and motivation

Elettra Sincontrone Trieste¹ is a big research facility that, principally, gives to users two light sources: the synchrotron Elettra and the laser at free electron Fermi. Besides this, it study new technology and technique to improve the their capabilities and work activities. A lot of the latest improvements are based on robots, that also after the covid and latest with the exploding diffusion of LLMs, allow an interaction of the users of the light sources to communicates and interact with the robots in the facility to obtain information and services as transport of packages. They already developed a global planner D-Star (D-star based optimized trajectory planner) [9] in ROS 1 for sending a robot in the infrastructure that is optimized for exploring a big space. But ROS 1 has reached the end of life so we need to rewrite the code using the ROS 2 ecosystem. Furthermore, because there are multiple vendor that are specialized for different type of work, there is the problem of the interaction between them. For this in the latest years is born Open-RMF² a multi platform for control and manage a multi fleet float of robots.

¹<https://www.elettra.eu/>

²<https://www.open-rmf.org/>

Chapter 2

Research questions and objectives

Our objects for this thesis is: given the company Mini RoverRobotics¹ Robot



Figure 2.1: Rover Mini of the RoverRobotics

import it to ROS 2 Nav2 ecosystem using a porting of the global planner D-star from ROS 1 to ROS 2 and then add it to Open-RMF to control it. Compare it to the ROS 1 stack and validate the use of the Open-RMF ecosystem in Elettra Sincrotrone Trieste.

¹<https://roverrobotics.com/>

Chapter 3

Scope and limitations

The principal limitation is the now non accessible control for the lifts that will not enable us to have a direct control on them and also there is not an automatic open-close control over the doors. Even if the hight customability of open-rmf is possible to use a human interaction for active these systems, like when the robot reach a door, it the send a message or a light can be turned on and then wait until someone will open the door or a lift. But is not a good approach.

Part II

Literature Review

In this part we will provide a description of the technologies used in this thesis. We will start describing what is ROS and how it works then we will talk about the navigation system used and how it is implemented the control with the fleet management system Open-RMF

Chapter 4

Overview of ROS1 and ROS2

The Robotics Operating System ¹ is a compilation of tools and libraries open-source that help the development of application in the robotic environment. otherwise from the name, it's not a Operating System (OS) but an Software Development Kit (SDK) that gives at the user all the tools for creating your robot giving some typically functionality of an OS: hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. The ROS ecosystem is based on (all these things is going to be described in detail in the next section):

- An communication system named middleware or plumbing based on DDS (Data Distribution Service). This allow the Nodes to share information between each other with an anonymous publish/subscribe pattern allowing fault isolation, separation of concerns and clear interfaces and a synchronous system (remote procedure call)RPC-style communication between services.
- Tools to control and improve the development of the software like launch, introspection, debugging, visualization, plotting, logging, and playback.
- Provides tools and libraries for obtaining, building, writing, and running code across multiple computers.
- A active and big community with at the center the Open Robotics².

4.1 Structure of ROS

The documentation³ give all the information of ROS, some functionality write below are not available for ROS 1:

- The **ROS Graph** is a peer-to-peer network of processes that elaborated simultaneously and communicate with each other.
- **Nodes** is a modular process and should be a specific resolution for a problem to be more easily communicate with other Nodes through topics, services, action and parameters. They also can be based on LifecycleNode that contain a state

¹<https://www.ros.org/>

²<https://www.openrobotics.org/>

³<https://docs.ros.org/en/rolling/index.html>

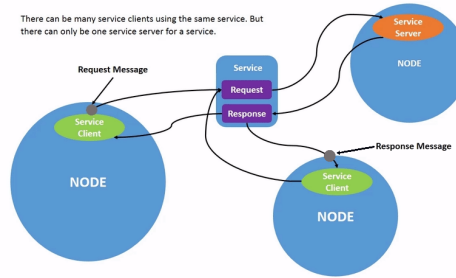


Figure 4.3: Service process

- **Parameters** are configuration of a node and allow to modified them without changing the code.
- **Actions** were created to fulfill the needs of something for tracking long task. They are a combination of two service (goal and result) and a pub-sub (feedback). It send a goal request and receive a response then send a result request, the node then start to pub to the feedback topic and then response to the result request.

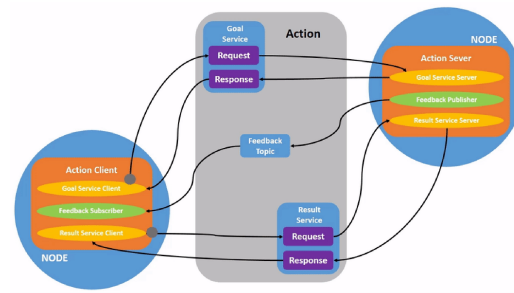


Figure 4.4: Action process

4.2 Tools of ROS

Now i will describe the tools that are built in ROS for developing robotics system:

- **Launch** is a file structure that allow to start multiple nodes with a set of parameters.
- **rqt** is a graphical user interface (GUI) tool for ROS 2. With a series of plugins allow to interact with an interface for helping in the development, in the monitoring and diagnostic the ROS system.
- **rqt_console** visualize the log generate by the ROS 2 system.
- **Rviz** is a 3D visualizer for the Robot Operating System framework?.
- **roscdep** is a dependency installer utility.
- **sros2** is a package that provide the tools and instruction to apply security on DDS.

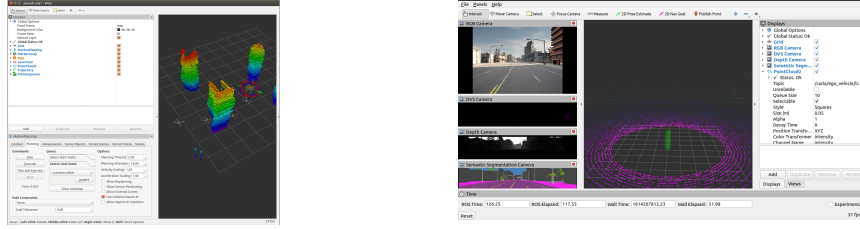


Figure 4.6: Examples of visualization of ROS data with Rviz

Another tools that are connected with ROS and helps develop is the simulation environments. They allow to test the code on a simulated robot in different condition to find possible bugs in the code that if there was launched on a real robot can be harmful to it or the environment. They provide realistic physical mechanics at all the parts of simulation and in the documentation of ROS there are listed 2: Gazebo and Webots. The first one is also a project under the Open Robotics and for a lot of projects is the default used like the one we will use. Gazebo offer:

- **ROS Integration** a bridge to converts the protobuf messages to ROS.
- **Performance tools** that allow to have it distributed on servers, dynamically asset loading and control of the time step for simulation to have the best performances.
- **Realistic simulation** that allow to insert a variety of sensor with their noise in a 3D rendering word built on Ogre 2.1⁴ and with the physic engine DART⁵.
- **Extensibility** because its built on modularity, it allow with plugins to run code that interact with the simulation.

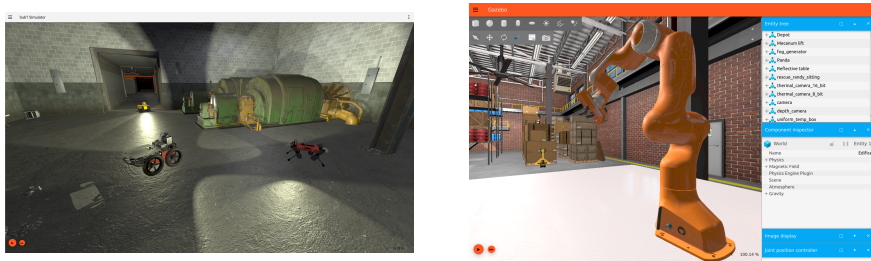


Figure 4.8: Examples of Gazebo

⁴<https://wiki.ogre3d.org/Home>

⁵<https://dartsim.github.io/>

Chapter 5

Introduction to NAV2 and Google Cartographer

Multiple tools are available for move a robot in an environment. The one that give a full system is Nav2 ¹ [6] the successor of ROS Navigation Stack of ROS 1.

5.1 Description of Nav2

Nav2 gives all the tools (perception, planning[8], control, localization, visualization) for whichever robot to navigate autonomously and reliably also thought hard environment. It allow to build a model of the environment from the data of the sensors, calculated dynamically the best path to follow to avoiding obstacles using a map, set the velocity of the motors and allow to follow logic step using a behaviors tree.

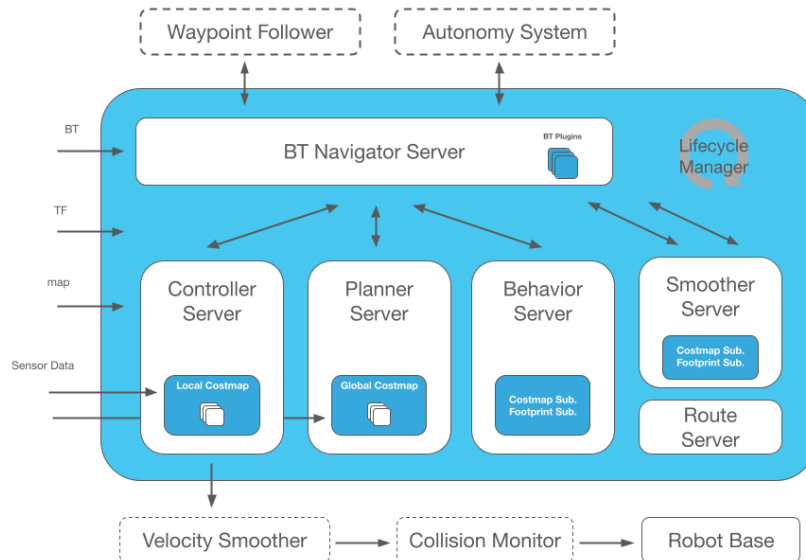


Figure 5.1: Nav2 architecture

These are also some of the updates that have changed from ROS 1 other less notable changes are the use of:

¹<https://nav2.org/>

- **Behavior Server** is the node that handles the behavior request like: spin, backup, wait.
- **Waypoint Follower** is a way/example for controlling the robot for doing an orchestrated task. This implementation take a list of waypoints, call the `NavigateToPose` action server and at each waypoint do a task (wait, take a photo, ...) as a plugin. This server can be changed with any **Autonomy System**
- **Route Server** for computing routes through a predefined navigation graph rather than using freespace planning like the Planner Server.
- **Velocity Smoother** as the name say smooth the velocity, acceleration and deadbands sent by Nav2 to the robot.
- **Costmap** is a representation of the environment using a regular 2D grid of cells with a cost (unknown, free, occupied, or inflated) and is used for generate the global plan or to compute local control efforts.

The next things that allow to use Nav2 is a:

- **state estimation** of the robot that generate a suitable TF tree, a tree structure of coordinate frames generated using the transform library[2], based on the REP 105, that is should provide this structure: `map` $\rightarrow$ `odom` $\rightarrow$ `base_link`. According to the REP 105 they are defines as:
 - `map` : a world fix frame that is not continuous that is the pose of the robot can change in discrete jumps at any time. This is useful as a long-term global reference but not in the short-term.
 - `odom` : a world fix frame that can drift over time to be continuous. This allow to be useful in an short-term local reference but not good for long-term one.
 - `base_link`: a robot fix frame.

The `odom` $\rightarrow$ `base_link` link can be calculated using different sensors. To increase the performance of our estimation we will use the `robot_localization` a node that provide nonlinear state estimation nodes using Kalman Filter [7]. The `map` $\rightarrow$ `odom` link can be generate with a global positioning system like Simultaneous Localization And Mapping (SLAM) method or using the one provided by Nav2, Adaptive Monte-Carlo Localization (AMCL).

- **map** that can be build during navigation using a SLAM tool or loaded. A powerful SLAM is Cartographer that will be described in the next section and it's the one used to build the map that we are going to use.

5.1.1 Cartographer

Cartographer[3] is a SLAM that work using two subsystem: a **Local SLAM** (called **Frontend**) that build a series of locally consistent **submaps** from successive scans and the pose extrapolator but with the possibility that can drift during acquisition

Chapter 6

Review of related work on porting navigation stack from ROS 1 to ROS 2

The big diffusion of ROS as a software for controlling robots bring new users cases that were not considered when was built. Starting as an academic project, the new cases that has met are the need of a way to control a fleet of them, deveoping on little embedded board and microcontrollers, real time system, not ideal network and the code was not designed for industrial usage. These changes made impossible to save ROS from a total rewrite of the code that brigh to have ROS 2, all more documentation is accessible at <https://design.ros2.org/> . Multiple company started using the ROS 2 ecosystem and how it increase their development[5] and also in the Nav2[8]. New projects are born like micro-ROS ¹ for controlling microcontrollers. While more detailed information about how to change the code are described in these sources, like the official documentation² and also secondary ³ and are the tools that are going to be used in the porting of the global planner. The notable changes, also described in that paper are:

¹<https://micro.ros.org/>

²<https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html>

³https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

Chapter 7

Overview of open source fleet management systems, particularly Open-RMF

7.1 Open-RMF description

...

today the tool for converting the data are <https://www.ros.org/blog/noetic-eol/>
implement also planning benchmark
describe and use composition in nav2 to obtain better performance [4]

Bibliography

- [1] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: <http://dx.doi.org/10.1201/9780429489105>.
- [2] Tully Foote. “tf: The transform library”. In: *Technologies for Practical Robot Applications (TePRA), 2013 IEEE International Conference on*. Open-Source Software workshop. 2013, pp. 1–6. DOI: 10.1109/TePRA.2013.6556373.
- [3] Wolfgang Hess et al. “Real-Time Loop Closure in 2D LIDAR SLAM”. In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278.
- [4] Steve Macenski et al. “Impact of ROS 2 Node Composition in Robotic Systems”. In: *IEEE Robotics and Automation Letters* 8.7 (2023), pp. 3996–4003. DOI: 10.1109/LRA.2023.3279614.
- [5] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [6] Steven Macenski et al. “The Marathon 2: A Navigation System”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.
- [7] T. Moore and D. Stouch. “A Generalized Extended Kalman Filter Implementation for the Robot Operating System”. In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.
- [8] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2”. In: *Robotics and Autonomous Systems* (2023).
- [9] Andrey Vukolov et al. “Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis”. In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.